Core java

By: Mr. Satyendra Singh

Department of Computer Science & Engineering,
ABESIT College of Engineering, Ghaziabad.

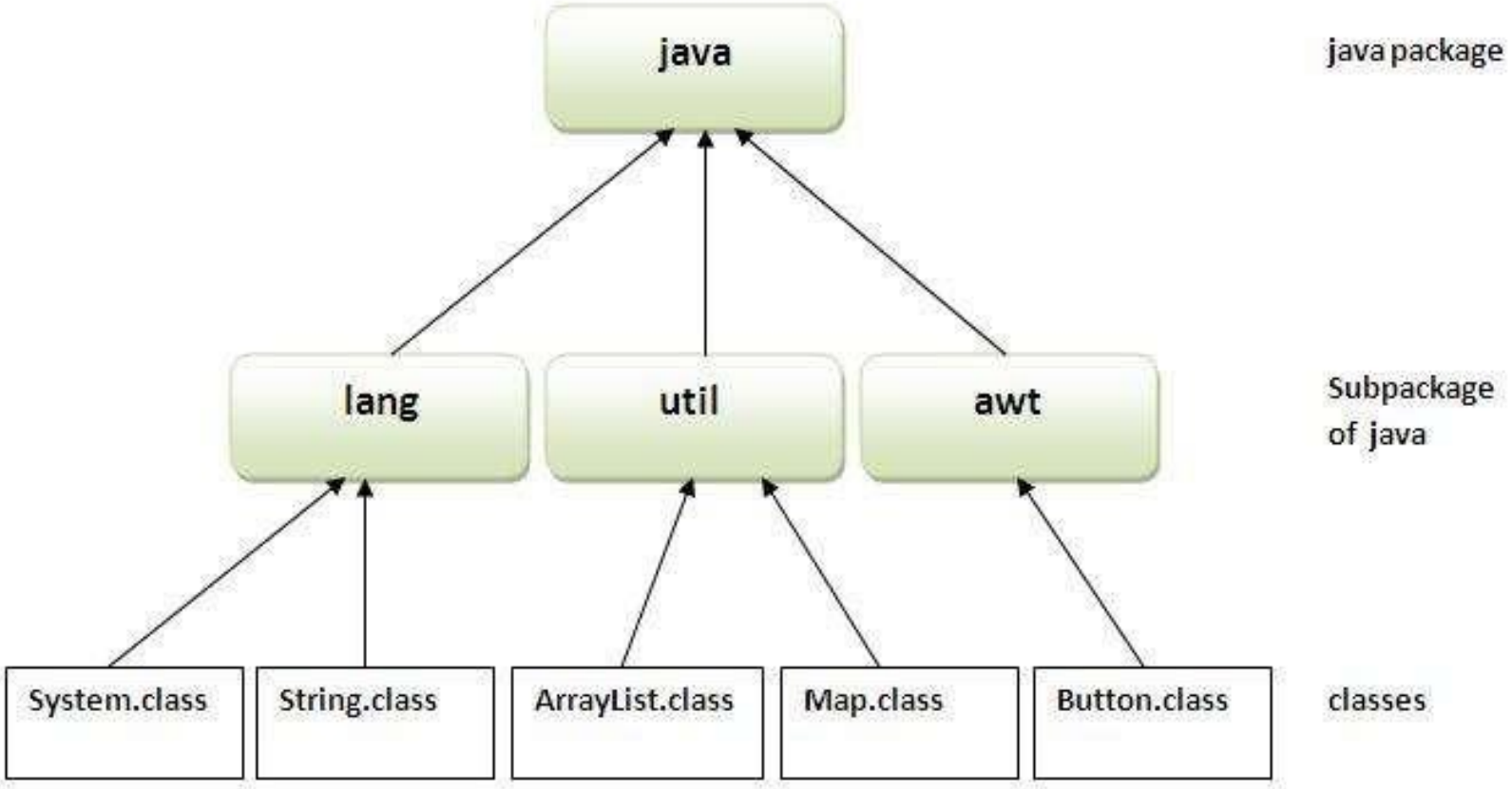
Package

1. Package
2. Example of package
3. Accessing package
4. By import packagename.*
5. By import packagename.classname
6. By fully qualified name
7. Subpackage

Package

- A package is a group of similar types of classs, interfaces and sub-packages.
- Package can be categorized in two form, built-in package and user-defined package. There are many built-in packages such as java, lang, awt, javax, swing, net, io, util, sql etc.
- In this page, we will have the detailed learning of creating user-defined packages.
- **Advantage of Package**
- Package is used to categorize the classes and interfaces so that they can be easily maintained.
- Package provids access protection & Package removes naming collision.

Package diagram



Example of Package

- The keyword package is used to create a package.
- //save as Simple.java package mypack;

```
public class Simple{
 public static void main(String args[]){
   System.out.println("Welcome to package");
  } }
```

To Compile: javac -d . Simple.java To Run: java mypack.Simple

Output:Welcome to package

The -d is a switch that tells the compiler where to put the class file i.e. it represents destination. The . represents the current folder.

How to access package from another package?

There are three ways to access the package from outside the package.

import package.*;

import package.classname;

fully qualified name.

- If you use package.* then all the classes and interfaces of this package will be accessible but not subpackages.
- The import keyword is used to make the classes and interface of another package accessible to the current package.

Example of package that import the packagename.*

Example of package that import the packagename.*

```
//save by A.java

package pack;

public class A{

 public void msg(){System.out.println("Hello");}

}

//save by B.java

package mypack;

import pack.*;

class B{    public static void main(String args[]){     A obj = new A();     obj.msg();    }  }
```

Output: Hello

Example

If you import package.classname then only declared class of this package will be accessible but not subpackages.

Example of package by import package.classname

```
//save by A.java
package pack;     public class A{   public void   msg(){System.out.println("Hello");}
}
//save by B.java
package mypack;  import pack.A;  class B{
  public static void main(String args[]){
   A obj = new A();
   obj.msg();    }    }
```

Output: Hello

Example of package by import fully qualified name

If you use fully qualified name then only declared class of this package will be accessible. Now there is no need to import. But you need to use fully qualified name every time when you are accessing the class or interface.

```
//save by A.java
package pack;  public class A{    public void msg(){System.out.println("Hello");}  }
//save by B.java
package mypack;
class B{
  public static void main(String args[]){
   pack.A obj = new pack.A();//using fully qualified name
   obj.msg();    } }
```

Output: Hello

Sequence of the program must be package then import then class.

- **Subpackage -** Package inside the package is called the subpackage.
- It should be created to categorize the package further. Let's take an example, Sun Microsystem has definded a package named java that contains many classes like System, String, Reader, Writer, Socket etc. These classes represent a particular group e.g. Reader and Writer classes are for Input/Output operation, Socket and ServerSocket classes are for networking etc and so on. So, Sun has subcategorized the java package into subpackages such as lang, net, io etc. and put the Input/Output related classes in io package, Server and ServerSocket classes in net packages and so on.

Example of subpackage

```
package com.
javatpoint.core;
class Simple{  public static void main(String args[]){   System.out.println("Hello
    subpackage");  }}
```

To Compile: javac -d . Simple.java

To Run: java com.javatpoint.core.Simple

Output: Hello subpackageNote:

If you import a package, all the classes and interface of that package will be imported excluding the classes and interfaces of the subpackages. Hence, you need to import the subpackage as well.

Questions ?